Chapter 19 worked solutions

Translation and Summarization

Exercises 1–4 work through fact preservation, training targets, decoding probabilities, and word overlap by hand. Exercises 9–11 check what an output says against its source and the task it was asked to perform.

1. Decide what must be preserved

A passenger wants to know why the last ferry ran late. A one-sentence summary could say, "The last ferry departed at 19:10 after a gate inspection delayed it." This keeps the reason for the delay and the actual departure time. The inspection answers the passenger's question, and the time tells the passenger when the ferry finally left. The number of passengers is incidental to that question.

A translation must carry over all the facts. It should identify the ferry as the last one, give its actual departure time as 19:10, say that it carried thirty passengers, and preserve the fact that the gate inspection caused the delay. The wording and time format may change to suit the target language, as long as the meaning is preserved.

2. Align two target lengths

For the target row $(5, 7, 6)$, the decoder first sees the beginning marker `BOS` and learns to predict 5. It then sees 5 and learns to predict 7, sees 7 and learns to predict 6, and sees 6 and learns to predict the end marker `EOS`. At each position, the decoder can use the preceding inputs as well as the current one. Giving it the correct previous tokens during training is called *teacher forcing*.

Prepend $BOS = 1$ to each decoder input and append $EOS = 2$ to each label row. The longer target needs four positions after adding its marker. Pad the shorter rows on the right with $PAD = 0$ to reach the same width.

$$Y_{in} = \begin{bmatrix} 1 & 5 & 7 & 6 \\ 1 & 8 & 9 & 0 \end{bmatrix}, \qquad Y_{target} = \begin{bmatrix} 5 & 7 & 6 & 2 \\ 8 & 9 & 2 & 0 \end{bmatrix}.$$

Read the two arrays column by column within each row. In the second row, for example, the decoder sees 9 at position three and learns to predict 2 there. The table marks every label used in the loss.

Target row	Position 1	Position 2	Position 3	Position 4
$(5, 7, 6)$	5 included	7 included	6 included	2 included
$(8, 9)$	8 included	9 included	2 included	0 ignored

There are $4 + 3 = 7$ supervised positions. Both end markers count because the model needs to learn when to stop.

3. Compare complete paths

One way to finish is to choose A and stop at the next step. A occurs with probability 0.7. Among sequences that start with A, the fraction that stop immediately is 0.6. The probability of following both steps is therefore $0.7(0.6) = 0.42$.

Apply the same multiplication to each complete sequence. A path that goes through C includes one more step, with conditional probability 1 for `EOS`.

Completed path	Product of step probabilities	Total probability
A → EOS	0.7(0.6)	0.42
A → C → EOS	0.7(0.4)(1)	0.28
B → EOS	0.3(0.9)	0.27
B → C → EOS	0.3(0.1)(1)	0.03

The four probabilities sum to 1. Greedy decoding chooses A because $0.7 > 0.3$. With A already chosen, it chooses `EOS` because $0.6 > 0.4$. The resulting path, A → EOS, also has the highest complete probability, 0.42.

4. Count repeated and ordered matches

The candidate *red red ferry departs* contains two copies of *red*. The reference *red ferry departs at noon* has only one copy, so it can match just one candidate occurrence. Each single token is a *unigram*. To clip a unigram's candidate count, take the smaller of its candidate and reference counts.

Candidate unigram	Candidate count	Reference count	Clipped count
red	2	1	1
ferry	1	1	1
departs	1	1	1

There are three matches. The denominator still counts all four candidate tokens, including both occurrences of *red*. The clipped unigram precision is therefore $(1 + 1 + 1)/4 = 3/4 = 0.75$.

For a common subsequence, match tokens in the same order in both sequences, skipping tokens when needed. One longest common subsequence is *red ferry departs*, of length three. Either candidate occurrence of *red* can match the reference's single *red*. A second match would require a second *red* in the reference. Since the remaining reference words *at* and *noon* never occur in the candidate, no longer match is possible.

9. Check a summary claim (optional)

The notice says that the ferry was late and that the north gate was closed. The candidate uses *because* to connect those events, so it makes three claims.

Candidate claim	Evidence in the notice
The ferry was late.	The notice states this directly.
The north gate was closed.	The notice states this directly.
The closure of the north gate caused the ferry to be late.	The notice leaves the cause of the delay open. The causal claim is unsupported.

Suppose the reference summary reads, “The ferry was late and the north gate was closed.” Using lowercase words as tokens and ignoring punctuation, both the reference and the candidate have ten tokens. Skip *and* in the reference and *because* in the candidate. The other nine tokens occur in the same order, giving a longest common subsequence of length nine. ROUGE-L precision, recall, and their harmonic mean are all $9/10 = 0.9$. In this example a single change adds a causal claim while leaving most of the reference wording intact. Checking that claim requires reading the notice.

10. Trace a missing fact (optional)

Suppose a passenger needs to know which gate the replacement ferry uses. Give the model the instruction *In one sentence, name the departure gate of the replacement ferry*. The answer-bearing sentence is *The replacement ferry departs from gate 7.* Other sentences can discuss the weather or ticket sales, with no other gate numbers. A successful output might be *The replacement ferry departs from gate 7.* We can now score a concrete fact even if the output uses different wording.

Set an encoder input limit of, for example, 128 tokens. Tokens are the pieces produced by the model’s tokenizer, so 128 tokens need not be 128 words. Build two versions of the same notice with the same instruction and filler. Swap the answer-bearing sentence with one irrelevant sentence. In the early version, put the entire answer-bearing sentence well within the limit. In the late version, put the entire sentence beyond the limit. Check the positions after formatting and tokenizing and adjust the filler if necessary. Keep the instruction at the front in both versions and use right truncation, which discards tokens at the end. Avoid putting the sentence across the boundary for this first comparison. A fragment such as *The replacement ferry departs from gate* would complicate the distinction between a visible and a missing fact.

Inspect the actual batch immediately before the encoder call. Record the formatted prompt, the tokenizer and maximum length, and the input ID row and its attention-mask row. An input ID identifies a token. In the usual binary attention mask, 1 marks a position the encoder may use and 0 marks padding. Decode the non-padding IDs and verify both that the instruction is intact and that the full gate sentence is visible in the early version and absent in the late one. Compare those positions against an untruncated tokenization of each version. Include any special tokens in the length check. This catches an error such as a second truncation step in batch preparation, even if the original document and an earlier tokenization both contained the fact.

Hold the checkpoint, prompt template, decoding method and output length cap fixed. Make the output cap long enough for the requested sentence. Check the output for the correct relation (*replacement ferry* departs from *gate 7*). Record an omission and a wrong gate separately. A guess of gate 7 when the sentence was removed matches the reference by chance or prior knowledge and lacks support in the visible input.

If the late input lacks the fact, preprocessing removed the opportunity to recover it from that source. If the early input contains the full instruction and fact and the output still omits it, the failure occurred after preprocessing, somewhere in encoding, conditional prediction or decoding. These observations alone cannot isolate which stage of the model failed. As a further check, shorten only the filler in the late notice so its answer sentence survives, verify the new encoder input, and repeat generation with the same settings. Recovery supports the truncation explanation. If the model also omits the fact in this control, both input visibility and model behavior need to be reported. Repeat with several notices and gate values to see whether the pattern holds.

11. Plan a comparison (optional)

Compare one compact pretrained encoder–decoder with a checkpoint obtained by fine-tuning those weights on notices paired with human-written summaries. A test input might report a ferry delay, its confirmed cause, and a new departure time. The intended summary includes the cause and the time. Both checkpoints receive the same input. This pairing lets us see, for example, whether fine-tuning recovers an omitted departure time at the cost of introducing a claim about the gate.

Group the corpus by underlying event. Keep all updates about one departure together and group duplicated or lightly rewritten notices before splitting. Assign complete groups to training, validation, and test, for example in approximate 70/15/15 proportions. Record the group IDs and counts in each split. Adapt using the training pairs only and use validation to choose the checkpoint. Hold test groups aside until the procedure is fixed. A random split by notice could let an almost identical incident appear in training and test, inflating the apparent ability to summarize new incidents.

Start each adaptation run from the same pretrained weights. Record the model and tokenizer revisions, prompt, seed, update budget, and saved checkpoints. For a concrete validation rule, evaluate each saved checkpoint with a fixed generation configuration and select the one with the highest mean validation ROUGE-L F1, taking the earlier checkpoint in a tie. Monitor source support on validation too, and report it separately. For the test comparison, freeze the same prompt, input truncation policy, beam width, length penalty, output cap, and end-token rule for both checkpoints. For example, fix width four and a 64-token output cap before opening the test set. A different decoder setting for each checkpoint would change two parts of the system at once.

Compute **ROUGE-L F1** against the human summary for each notice under one fixed tokenization and average those per-notice scores. The measure uses the longest subsequence that appears in both summaries in the same order, then balances precision and recall for that subsequence. A summary that rephrases the facts may get a lower score. A fluent summary can also retain a high overlap score while introducing an unverified cause. We therefore need a separate check against the notice.

Before reading either model's outputs, list the required source facts for each test notice. For each summary, mark which of those facts it correctly conveys. Next, split its factual statements into individual claims and mark each as supported, contradicted, or unverified using the source. For example, if the notice confirms a 19:10 departure but says nothing about a gate inspection, *leaves at 19:10 because of a gate inspection* conveys the time but introduces an unverified cause. Annotate both checkpoints under the same written rules with checkpoint identity hidden, and resolve disagreements against the notice.

Report coverage as required facts conveyed divided by required facts listed. Report unsupported additions separately as the count and share of generated factual claims that are contradicted or unverified, with the two categories visible. For example, two required facts conveyed out of three give coverage of 2/3. If the summary also contains one unverified claim and no contradictions among its four factual claims, the unsupported share is 1/4. Include raw counts and the fraction of notices containing at least one unsupported addition. For the overall coverage and unsupported shares, add the respective numerators and denominators across the test notices before dividing. An empty summary gives no coverage on a notice with required facts and has no claim-level denominator, so record empty outputs separately. A model can improve one measure and worsen the other.

Show results side by side and report paired differences on the same test notices. If estimating uncertainty, resample by event group and retain both outputs for every notice in each sampled group. This preserves the dependence among repeated notices and the pairing between systems. The result describes adaptation under the recorded data, generation settings and annotation policy.